

AstraPay Capstone: SQL Deliverables

Task 4: Trusted Profitability Data Model

File: 04_trusted_profitability_model.sql

Description: This is the core transformation query. It cleans invalid amounts, pre-aggregates the one-to-many operational tables (Events, Fraud, Settlements) to the transaction grain to prevent data fan-out, strictly joins the **SETTLEMENT** FX rate, and calculates final contribution profit.

SQL

```
/*
=====

ASTRAPAY: TRUSTED PROFITABILITY MODEL

Level: Core Analytical Base Table (ABT)

Grain: 1 Row = 1 Payment Attempt

=====
*/

-- 1. Clean Base Transactions (Removes 0 or negative amount defects)

WITH clean_transactions AS (

    SELECT

        transaction_id,

        customer_id,

        merchant_id,

        route_id,

        amount,

        currency,

        status,
```

```
        DATE(created_at) AS txn_date,  
        STRFTIME('%Y-%m', created_at) AS txn_month  
FROM "transaction"  
WHERE amount > 0  
,
```

-- 2. Pre-Aggregate One-to-Many Traps to the Transaction Grain

```
agg_fraud AS (  
    SELECT  
        transaction_id,  
        MAX(risk_score) AS max_risk_score  
    FROM fraud_decision  
    GROUP BY transaction_id  
,
```

```
agg_events AS (  
    SELECT  
        transaction_id,  
        SUM(processing_ms) AS total_processing_ms  
    FROM payment_event  
    GROUP BY transaction_id  
,
```

```
agg_settlement AS (  
    SELECT  
        transaction_id,
```

```
        SUM(settlement_amount) AS total_settlement_amount,  
        SUM(fee_amount) AS total_settlement_fee  
FROM settlement  
GROUP BY transaction_id  
,
```

```
agg_chargeback AS (  
    SELECT  
        transaction_id,  
        SUM(amount) AS total_chargeback_amount  
FROM chargeback  
GROUP BY transaction_id  
,
```

-- 3. Filter for the specific SETTLEMENT FX Rate

```
fx_settlement_rates AS (  
    SELECT  
        rate_date,  
        currency,  
        rate_to_usd  
FROM fx_rate  
WHERE rate_type = 'SETTLEMENT'  
)
```

-- 4. Final Merged Model & Economics Calculation

```
SELECT
```

```

t.transaction_id,
t.txn_date,
t.txn_month,
t.status,
t.currency,
t.amount AS amount_local,

-- FX Conversion to USD
(t.amount * fx.rate_to_usd) AS gpv_usd,

-- Gross Revenue (Merchant Pricing Extraction)
CASE
    WHEN m.pricing_plan = 'STANDARD_2.0%' THEN (t.amount * fx.rate_to_usd) * 0.020
    WHEN m.pricing_plan = 'ENTERPRISE_1.2%' THEN (t.amount * fx.rate_to_usd) *
0.012
    WHEN m.pricing_plan = 'CUSTOM_TIER' THEN (t.amount * fx.rate_to_usd) * 0.015
    ELSE 0
END AS gross_revenue_usd,

-- Processing Costs (Fixed + Variable)
(r.fixed_fee + (t.amount * fx.rate_to_usd * r.variable_fee_pct)) AS
route_processing_cost_usd,

-- Attributed Operational Costs
COALESCE((s.total_settlement_fee * fx.rate_to_usd), 0) AS settlement_fee_usd,
COALESCE((cb.total_chargeback_amount * fx.rate_to_usd), 0) AS chargeback_loss_usd,

```

-- Final Contribution Profit (Only calculated for SUCCESSful and Settled txns)

CASE

WHEN t.status = 'SUCCESS' AND s.total_settlement_amount IS NOT NULL THEN

(

(CASE

0.020 WHEN m.pricing_plan = 'STANDARD_2.0%' THEN (t.amount * fx.rate_to_usd) *

fx.rate_to_usd) * 0.012

0.015 WHEN m.pricing_plan = 'CUSTOM_TIER' THEN (t.amount * fx.rate_to_usd) *

ELSE 0

END)

- (r.fixed_fee + (t.amount * fx.rate_to_usd * r.variable_fee_pct))

- COALESCE((s.total_settlement_fee * fx.rate_to_usd), 0)

- COALESCE((cb.total_chargeback_amount * fx.rate_to_usd), 0)

)

ELSE 0

END AS contribution_profit_usd,

-- Dimensional Attributes for Slicing

m.merchant_id,

m.pricing_plan,

m.category AS merchant_category,

m.country AS merchant_country,

c.segment AS customer_segment,

t.route_id,

r.provider AS route_provider,

```
f.max_risk_score,  
e.total_processing_ms
```

```
FROM clean_transactions t  
  
LEFT JOIN merchant m ON t.merchant_id = m.merchant_id  
  
LEFT JOIN customer c ON t.customer_id = c.customer_id  
  
LEFT JOIN route_cost r ON t.route_id = r.route_id  
  
LEFT JOIN fx_settlement_rates fx ON t.currency = fx.currency AND t.txn_date = fx.rate_date  
  
LEFT JOIN agg_fraud f ON t.transaction_id = f.transaction_id  
  
LEFT JOIN agg_events e ON t.transaction_id = e.transaction_id  
  
LEFT JOIN agg_settlement s ON t.transaction_id = s.transaction_id  
  
LEFT JOIN agg_chargeback cb ON t.transaction_id = cb.transaction_id;
```

Task 5: Root-Cause Investigation Queries

File: `05_diagnostic_queries.sql`

Description: These queries execute against the finalized `trusted_profitability_model` to mathematically identify the specific operational and commercial drivers of the margin collapse.

SQL

```
/*  
=====
```

DIAGNOSTIC QUERY 1: Overall Temporal Deterioration (Latency vs. Margin)

Purpose: Identifies when the margin drop occurred and correlates it with the massive spike in processing latency.

```
=====
```

*/

SELECT

```

    txn_month,
    COUNT(transaction_id) AS success_volume,
    ROUND(AVG(contribution_profit_usd), 2) AS avg_profit_per_txn,
    ROUND(AVG(total_processing_ms), 0) AS avg_processing_ms
FROM trusted_profitability_model
WHERE status = 'SUCCESS'
    AND settlement_fee_usd > 0
GROUP BY txn_month
ORDER BY txn_month;

```

```

/*
=====

```

DIAGNOSTIC QUERY 2: Commercial Mix-Shift & Route Deterioration

Purpose: Tracks volume shifting from low-cost primary routes to
high-cost secondary routes following the latency failures.

```

=====
*/

```

```

SELECT
    txn_month,
    route_id,
    COUNT(transaction_id) AS volume,
    ROUND(AVG(contribution_profit_usd), 2) AS avg_profit
FROM trusted_profitability_model
WHERE status = 'SUCCESS'
GROUP BY
    txn_month,

```

```

route_id
ORDER BY

route_id,

txn_month;

/*
=====

DIAGNOSTIC QUERY 3: Merchant Pricing Impact

Purpose: Proves that the STANDARD_2.0% fixed pricing tier mathematically

fails to cover costs when transactions fall back to expensive routes.

=====
*/

SELECT

txn_month,

merchant_category,

pricing_plan,

COUNT(transaction_id) AS volume,

ROUND(AVG(contribution_profit_usd), 2) AS avg_profit

FROM trusted_profitability_model

WHERE status = 'SUCCESS'

GROUP BY

txn_month,

merchant_category,

pricing_plan

ORDER BY

volume DESC,

```

txn_month

LIMIT 10